CÓDIGOCLARO · RUTA GESTIÓN DE BASES DE DATOS

Gestión de bases de datos con PostgreSQL

15 temas · modelado, administración, seguridad y recuperación

Las prácticas se contrastan con la documentación oficial de PostgreSQL 18; revisa la versión de tu instalación antes de aplicarlas.

Tema 01 · Requisitos y modelo de datos

Objetivo

Traducir necesidades operativas en reglas de datos antes de escoger tablas, tipos o índices.

Explicación

Empieza por quién registra y consulta datos, qué eventos deben conservarse, qué relaciones existen y qué significa que un dato sea válido. Distingue el modelo conceptual y lógico de las decisiones físicas de almacenamiento y rendimiento.

Usaremos una biblioteca de práctica. La regla de que un ejemplar solo puede tener un préstamo activo se asume para el ejercicio; una organización real debe confirmar sus políticas, retención y recuperación esperada con sus responsables.

Ejemplo guiado

Ficha de requisitos antes del DDL · modelo didáctico

```
Entidades: Socio, Ejemplar, Prestamo
Evento: apertura y devolucion de un prestamo
Regla asumida: un ejemplar tiene como maximo
            un prestamo activo a la vez
Pendiente: confirmar plazos y datos que se retienen
```

Lectura

Cada regla debe convertirse en una decisión que pueda revisarse: una relación, una restricción, una transacción o un procedimiento operativo. Separa los requisitos confirmados de las hipótesis del ejercicio.

Alcance y precisión

El modelado lógico no decide por sí solo el motor, el particionado, la estrategia de backup ni los controles de privacidad. Esas decisiones se contrastan con carga, plataforma y requisitos de la organización.

Práctica

Entrevista a un usuario hipotético de biblioteca. Anota actores, búsquedas, cambios permitidos, excepciones y cuánto tiempo debe conservarse cada dato.

Comprobación

Puedes señalar qué afirmaciones son requisitos confirmados y qué preguntas siguen abiertas antes de crear el esquema.

Fuentes enlazadas: Diseño lógico y físico · Oracle · Definición de datos · PostgreSQL

Tema 02 · Modelo entidad-relación

Objetivo

Representar entidades, atributos, identificadores y cardinalidades antes de transformarlos en tablas.

Explicación

Una entidad representa algo sobre lo que se conserva información; sus atributos describen ese objeto y una relación expresa cómo se asocia con otros. En el caso de préstamos, un socio puede tener varios préstamos a lo largo del tiempo y cada préstamo se refiere a un socio y un ejemplar.

La cardinalidad describe cuántos elementos pueden relacionarse. La regla temporal de un solo préstamo activo por ejemplar exige una restricción adicional: no queda expresada únicamente con el dibujo 1:N.

Ejemplo guiado

Relaciones del caso de biblioteca · esquema conceptual

```
SOCIO    (1) ---- (N) PRESTAMO
EJEMPLAR (1) ---- (N) PRESTAMO
Cada PRESTAMO apunta a un socio y un ejemplar
Regla aparte: 0..1 prestamo activo por ejemplar
```

Lectura

En el modelo relacional, la clave foránea suele ubicarse en el lado N: prestamo guarda referencias al socio y al ejemplar. Las cardinalidades y opcionalidades deben corresponder a reglas del dominio, no a una conveniencia del diagrama.

Alcance y precisión

El dibujo es una notación textual abreviada, no un diagrama ER completo. Antes de implantarlo, aclara participación opcional, historial, borrados y si los identificadores son naturales o sustitutos.

Práctica

Añade la entidad Sede si los ejemplares pertenecen a una sucursal. Especifica si una sede puede existir sin ejemplares y qué relación expresa esa regla.

Comprobación

Puedes leer cada relación en ambos sentidos y justificar dónde quedará cada referencia al pasar de entidades a tablas.

Fuentes enlazadas: Modelado entidad-relación · Oracle · Claves foráneas · PostgreSQL

Tema 03 · Normalización: 1FN, 2FN y 3FN

Objetivo

Detectar grupos repetidos y dependencias que generan duplicación y anomalías al insertar, cambiar o borrar datos.

Explicación

En una introducción relacional, 1FN evita grupos repetidos y guarda un valor por atributo. 2FN requiere 1FN y elimina dependencias parciales respecto de una clave candidata compuesta. 3FN reduce dependencias transitivas de atributos no clave sobre la clave.

Ejemplo: si id_socio determina nombre_socio, guardar el nombre en cada préstamo repite el mismo hecho. Separa Socio(id_socio, nombre) de Prestamo(id_prestamo, id_socio, fecha). La normalización reduce anomalías, aunque el diseño debe considerar también las consultas y su carga.

Ejemplo guiado

Señales para descomponer una tabla · conceptos relacionales

```
1FN: sin columnas repetidas ni listas en una celda
2FN: atributos no clave dependen de la clave completa
     (importa cuando la clave candidata es compuesta)
3FN: separa dependencias transitivas no clave
Socio(id_socio, nombre) <- Prestamo(id_socio, fecha)
```

Lectura

La última línea indica que Prestamo conserva la clave de Socio, mientras el nombre vive una sola vez en Socio. En una tabla de líneas con clave (pedido, numero_linea), un nombre de producto que depende solo de producto_id es una señal de dependencia parcial.

Alcance y precisión

Las definiciones introductorias pueden simplificarse; la teoría formal se expresa con claves candidatas y dependencias funcionales. No desnormalices por intuición: identifica primero una necesidad medida y conserva una fuente de verdad clara.

Práctica

Revisa una tabla Pedido(pedido_id, cliente_id, nombre_cliente, fecha, producto_id, nombre_producto, cantidad). Propón descomposición y señala qué atributo depende de qué clave.

Comprobación

Puedes describir al menos una anomalía que evita cada separación y explicar por qué 2FN exige atención especial con claves compuestas.

Fuentes enlazadas: Normalización · Oracle Database 26 · Restricciones y claves · PostgreSQL

Tema 04 · Claves primarias y foráneas

Objetivo

Identificar filas de forma estable y enlazar tablas con integridad referencial.

Explicación

Una clave primaria identifica una fila y no admite duplicados ni NULL. Una clave foránea exige que cada valor referenciado tenga una fila correspondiente en la tabla destino, salvo cuando la relación es opcional y la columna admite NULL.

PostgreSQL crea un índice para la clave primaria, pero no crea automáticamente un índice en todas las columnas que referencian una clave foránea. El índice hijo depende de las consultas y del coste de actualizar o borrar filas padre.

Ejemplo guiado

Dos tablas vinculadas · PostgreSQL 18

```
CREATE TABLE socio (
  id_socio bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  nombre text NOT NULL
);
CREATE TABLE prestamo (
  id_prestamo bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  id_socio bigint NOT NULL REFERENCES socio(id_socio),
  fecha_prestamo date NOT NULL DEFAULT CURRENT_DATE
```

```
);
```

Lectura

La segunda tabla no puede almacenar un id_socio inexistente. La columna REFERENCES apunta a la clave de socio; PRIMARY KEY también declara unicidad y ausencia de NULL. DEFAULT CURRENT_DATE completa fecha_prestamo al insertar si se omite, usando la fecha de inicio de la transacción.

Alcance y precisión

La sintaxis del ejemplo usa columnas identity de PostgreSQL. Otros motores y versiones ofrecen mecanismos de generación de claves con sintaxis y detalles distintos.

Práctica

Decide si id_ejemplar debe ser clave natural o sustituta. Añade la relación de Prestamo con Ejemplar y analiza qué debe pasar si se intenta borrar un ejemplar con historial.

Comprobación

Prueba mentalmente un duplicado de clave primaria, un NULL y una referencia inexistente; puedes explicar qué regla rechaza cada caso.

Fuentes enlazadas: Primary keys y foreign keys · PostgreSQL · Tutorial de claves foráneas · PostgreSQL · Valores por defecto · PostgreSQL · Fecha y hora actuales · PostgreSQL

Tema 05 · Restricciones de integridad

Objetivo

Expresar reglas locales de validez en el esquema y distinguirlas de reglas que requieren coordinación entre filas.

Explicación

NOT NULL exige un valor; UNIQUE impide duplicados; CHECK evalúa una condición de la fila; una clave foránea protege referencias. Poner la regla cerca de los datos evita que una ruta de escritura alternativa la omita.

En PostgreSQL, un CHECK que evalúa a NULL no rechaza la fila. Por eso una columna obligatoria suele requerir además NOT NULL. Un CHECK no es el mecanismo para garantizar condiciones que dependen de otras filas o tablas.

Ejemplo guiado

Restringir valores de ejemplar · PostgreSQL 18

```
CREATE TABLE ejemplar (
  id_ejemplar bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  codigo text NOT NULL UNIQUE,
  estado text NOT NULL CHECK
    (estado IN ('disponible', 'baja'))
);
```

Lectura

La clave primaria identifica la fila; el código debe estar presente y ser único; el estado solo puede tomar uno de los dos valores enumerados. Si estado omitiera NOT NULL, el CHECK por sí solo permitiría NULL.

Alcance y precisión

La lista de estados es una política de ejemplo. CHECK en PostgreSQL está pensado para la fila actual; usa claves foráneas o restricciones únicas para relaciones y unicidad, y diseña transacciones para reglas concurrentes más amplias.

Práctica

Añade fecha_alta NOT NULL y una condición que impida una fecha posterior a la fecha de hoy solo si la regla debe ser evaluada al insertar; analiza qué pasa si el día cambia después.

Comprobación

Cada regla tiene un mecanismo adecuado y sabes identificar el caso NULL que una condición CHECK podría dejar pasar.

Fuentes enlazadas: CHECK, NOT NULL, UNIQUE y claves · PostgreSQL · CREATE TABLE · PostgreSQL

Tema 06 · Transacciones y propiedades ACID

Objetivo

Agrupar cambios que forman una sola operación y saber cuándo confirmar o revertirlos.

Explicación

Una transacción hace que varios pasos se confirmen como una unidad o se reviertan si la operación no puede completarse. ACID resume atomicidad, consistencia, aislamiento y durabilidad; las garantías concretas dependen también del motor, su configuración y el nivel de aislamiento.

Una transferencia actualiza dos cuentas. Si solo se guarda el débito o solo el abono, el resultado es incorrecto. BEGIN abre el trabajo, COMMIT lo confirma y ROLLBACK solicita deshacer los cambios pendientes.

Ejemplo guiado

Agrupar dos actualizaciones · PostgreSQL 18

```
BEGIN;
UPDATE cuenta SET saldo = saldo - 20 WHERE id = 1;
UPDATE cuenta SET saldo = saldo + 20 WHERE id = 2;
COMMIT;
-- Si la operacion no es valida: ROLLBACK;
```

Lectura

El ejemplo muestra el límite transaccional, no toda la lógica de una transferencia. La aplicación debe comprobar que existen las cuentas, que el saldo es suficiente y que el número de filas actualizado coincide con lo esperado.

Alcance y precisión

Una transacción no arregla por sí sola una regla de negocio mal definida. La durabilidad y el aislamiento son garantías del motor bajo sus condiciones documentadas, no sustituyen backups ni tratamiento de errores.

Práctica

Simula una biblioteca: inserta el préstamo y cambia la disponibilidad del ejemplar en la misma transacción. Decide qué hacer si el ejemplar ya no está disponible.

Comprobación

Puedes indicar qué cambios se revierten si falla el segundo paso y qué comprobaciones necesita el caso de uso antes de confirmar.

Fuentes enlazadas: Transacciones · PostgreSQL · Aislamiento de transacciones · PostgreSQL

Tema 07 · Bloqueos y concurrencia

Objetivo

Coordinar transacciones que compiten por una misma fila sin asumir que las lecturas serializan los cambios.

Explicación

PostgreSQL usa MVCC para que las sesiones puedan leer versiones consistentes mientras otras transacciones trabajan. Cuando una regla exige tomar una fila y modificarla como unidad, SELECT FOR UPDATE puede bloquear cambios incompatibles hasta que termine la transacción.

Los bloqueos duran hasta COMMIT o ROLLBACK. Mantén la transacción corta: esperas largas reducen concurrencia y bloqueos cruzados pueden producir deadlocks, que el motor detecta y resuelve abortando una transacción.

Ejemplo guiado

Reservar un ejemplar con bloqueo de fila · PostgreSQL 18

```
BEGIN;
SELECT disponible FROM ejemplar
WHERE id_ejemplar = 7 FOR UPDATE;
-- La aplicacion valida que disponible sea true
UPDATE ejemplar SET disponible = false
WHERE id_ejemplar = 7;
COMMIT;
```

Lectura

La fila seleccionada queda protegida frente a otros escritores y sesiones que intenten bloquearla con modos incompatibles. Las lecturas ordinarias no quedan bloqueadas por un row lock; la aplicación debe tratar el caso de que no haya fila.

Alcance y precisión

El ejemplo muestra una secuencia educativa y omite el control de errores del cliente. Un bloqueo explícito no reemplaza una restricción de integridad ni una estrategia de reintento ante deadlocks o fallos de serialización.

Práctica

Abre dos sesiones sobre el mismo ejemplar y describe cuál espera. Repite la operación sobre ejemplares distintos y observa por qué un bloqueo a nivel de fila permite más concurrencia que bloquear toda la tabla.

Comprobación

Sabes cuándo se libera el bloqueo, qué operaciones pueden esperar y por qué una transacción no debe quedarse abierta mientras el usuario decide.

Fuentes enlazadas: Control de concurrencia y MVCC · PostgreSQL · Bloqueos explícitos y deadlocks · PostgreSQL

Tema 08 · Índices y planes de ejecución

Objetivo

Relacionar un índice con un patrón de consulta y revisar el plan que el optimizador eligió.

Explicación

Un índice puede ayudar a encontrar filas o producir un orden sin recorrer toda la tabla, pero consume espacio y añade trabajo a inserciones, cambios y borrados. El optimizador decide si le conviene según consulta, distribución de datos y estadísticas.

EXPLAIN muestra el plan estimado. EXPLAIN ANALYZE ejecuta la consulta y añade medidas reales; úsalo aquí con un SELECT de solo lectura y entiende primero sus efectos si lo aplicas a una sentencia que modifica datos.

Ejemplo guiado

Índice y diagnóstico de una consulta · PostgreSQL 18

```
CREATE INDEX ix_prestamo_socio_fecha
  ON prestamo (id_socio, fecha_prestamo);
EXPLAIN (ANALYZE, BUFFERS)
SELECT id_prestamo FROM prestamo
WHERE id_socio = 42
ORDER BY fecha_prestamo DESC;
```

Lectura

El índice compuesto empieza por id_socio y luego fecha_prestamo, coherente con el filtro y orden del ejemplo. No garantiza que el planificador lo use: inspecciona filas estimadas y reales antes de decidir si el índice aporta.

Alcance y precisión

Los costes y planes dependen del conjunto de datos, estadísticas, configuración y hardware. No copies una recomendación de índice sin comparar consultas representativas y el coste de escritura.

Práctica

Compara EXPLAIN para una consulta por socio con y sin el índice. Guarda el plan y explica qué nodo cambia; evita medir sobre datos diminutos como si fueran carga real.

Comprobación

Puedes distinguir un plan estimado de una ejecución medida y justificar qué consulta concreta pretende acelerar el índice.

Fuentes enlazadas: Índices · PostgreSQL · EXPLAIN y EXPLAIN ANALYZE · PostgreSQL

Tema 09 · Usuarios, roles y privilegios

Objetivo

Separar propiedad de objetos, identidad de conexión y permisos concedidos a cada función.

Explicación

En PostgreSQL, los roles pueden representar usuarios, grupos o ambos. El propietario de un objeto tiene facultades propias; los demás reciben permisos explícitos con GRANT y pueden perderlos con REVOKE.

Asigna al rol de una aplicación solo los permisos que necesita. Para usar una tabla dentro de un esquema hacen falta permisos adecuados en el esquema y en el objeto; USAGE sobre el esquema no concede por sí solo lectura de la tabla.

Ejemplo guiado

Permiso de lectura limitado · PostgreSQL 18

```
CREATE ROLE informes NOLOGIN;
GRANT USAGE ON SCHEMA biblioteca TO informes;
GRANT SELECT ON biblioteca.prestamo TO informes;
-- Sin INSERT, UPDATE ni DELETE
```

Lectura

El rol puede resolver objetos del esquema y leer la tabla indicada, pero no recibe permisos de escritura por estas instrucciones. Un rol NOLOGIN puede servir como grupo de privilegios que luego se concede a roles de conexión.

Alcance y precisión

El ejemplo no revoca permisos preexistentes ni configura acceso a la base de datos o secuencias. Revisa privilegios predeterminados, membresías y propiedad de objetos al desplegar en un entorno real.

Práctica

Diseña tres roles: propietario de migraciones, aplicación de escritura y analista de solo lectura. Enumera qué tablas y operaciones necesita cada uno.

Comprobación

Puedes explicar por qué USAGE en un esquema no equivale a SELECT y qué acciones evita cada privilegio omitido.

Fuentes enlazadas: Roles de base de datos · PostgreSQL · Privilegios · PostgreSQL · GRANT · PostgreSQL

Tema 10 · Copias de seguridad

Objetivo

Elegir una estrategia de backup de acuerdo con el alcance de recuperación y conservar una copia verificable.

Explicación

PostgreSQL describe tres enfoques distintos: volcado SQL, copia del sistema de archivos y archivado continuo de WAL. pg_dump exporta una base de datos de forma consistente mientras se usa; no incluye por sí solo todos los objetos globales del clúster, como roles.

Una política debe definir frecuencia, retención, ubicación separada del servidor, cifrado y objetivos de recuperación. El resultado no se considera suficiente hasta que se ha probado una restauración y se han revisado sus permisos y contenido.

Ejemplo guiado

Volcado lógico en formato de archivo · PostgreSQL 18

```
pg_dump -Fc -f biblioteca.dump biblioteca
# -Fc crea un archivo de formato custom
# El destino y la retencion se definen aparte
```

Lectura

El formato custom se restaura con pg_restore y permite seleccionar objetos. El comando se limita a una base; revisa pg_dumpall u otro procedimiento para roles y demás objetos globales.

Alcance y precisión

Un volcado lógico puede no ser la estrategia adecuada para backups regulares de producción de gran tamaño. Compara recuperación lógica, copia física y PITR con la documentación y el tamaño real del servicio.

Práctica

Escribe un calendario de backup para una base de práctica: qué se copia, a qué medio separado, quién puede leerlo y qué prueba periódica confirmará que el archivo es utilizable.

Comprobación

Puedes distinguir un backup por base de uno del clúster y explicar cómo probarías una copia sin sobrescribir el origen.

Fuentes enlazadas: Estrategias de backup y restore · PostgreSQL · pg_dump · PostgreSQL

Tema 11 · Restauración y recuperación

Objetivo

Diferenciar restaurar un archivo lógico de reconstruir un servidor y recuperar hasta un punto en el tiempo.

Explicación

Un archivo pg_dump en formato custom se reconstruye con pg_restore. PITR requiere una copia base compatible y una secuencia archivada de WAL; no se obtiene solo porque exista un volcado SQL.

Una prueba útil restaura en un destino aislado, comprueba tablas y conteos relevantes, ejecuta una consulta de negocio y documenta cuánto tardó. Restaurar sobre el único original sin validar el objetivo puede aumentar el daño.

Ejemplo guiado

Ensayo de restauración en una base nueva · PostgreSQL 18

```
createdb biblioteca_prueba
pg_restore --no-owner --no-privileges -d biblioteca_prueba biblioteca.dump
# Verificar objetos y consultas antes de aceptar
```

Lectura

El ejemplo presupone que biblioteca.dump se creó con pg_dump -Fc. La opción omite propietarios y ACL del archivo, lo que evita exigir que existan esos roles durante el ensayo; el entorno final puede requerir restaurar también permisos.

Alcance y precisión

Para PITR, define un objetivo temporal alcanzable después del fin de la copia base y conserva la cadena necesaria de WAL. El procedimiento exacto incluye configuración del servidor y no se reduce a pg_restore.

Práctica

Restaura el archivo de práctica en otra base y redacta un acta con versión del motor, hora, errores, conteos y consulta de aceptación; no ejecutes el ensayo sobre datos de producción.

Comprobación

Puedes decidir si una incidencia exige restaurar objetos de un dump, una base completa o una copia base más WAL hasta un instante objetivo.

Fuentes enlazadas: pg_restore · PostgreSQL · Recuperación PITR · PostgreSQL · Backup y restauración · PostgreSQL

Tema 12 · Migraciones de esquema

Objetivo

Cambiar el esquema con pasos versionados, revisables y compatibles con los datos y la aplicación.

Explicación

Una migración es un cambio controlado del esquema y, a veces, de los datos. Guarda cada paso con versión y una descripción; revisa precondiciones, orden, reversión posible, ventana de ejecución, backup y compatibilidad con las versiones de la aplicación que estarán activas.

En PostgreSQL, ALTER TABLE tiene subformas con niveles de bloqueo distintos. Añadir una restricción puede recorrer filas existentes; una migración pequeña no siempre tiene coste pequeño en una tabla grande.

Ejemplo guiado

Migración aditiva y acotada · PostgreSQL 18

```
BEGIN;
ALTER TABLE prestamo
  ADD COLUMN fecha_devolucion date;
COMMIT;
```

Lectura

El cambio agrega una columna opcional y no borra información existente. La aplicación debe tolerar que esa columna esté vacía durante el despliegue; el control externo debe registrar qué versión de migración quedó aplicada.

Alcance y precisión

No todas las operaciones de DDL tienen el mismo bloqueo, duración o reversibilidad. Inspecciona la forma exacta en el manual de la versión destino y prueba con volumen y concurrencia representativos.

Práctica

Añade estado_devuelto en una migración nueva. Planifica cómo desplegar primero el lector tolerante, rellenar datos antiguos y luego imponer una restricción sin interrumpir el servicio.

Comprobación

Puedes describir el estado del esquema antes y después, cómo detectar si el paso ya se aplicó y qué riesgo operativo tiene la instrucción.

Fuentes enlazadas: ALTER TABLE y bloqueos · PostgreSQL · Definición y restricciones · PostgreSQL

Tema 13 · Auditoría, logs y trazabilidad

Objetivo

Registrar eventos operativos útiles sin tratar cualquier log SQL como un historial de auditoría completo.

Explicación

Los logs de PostgreSQL pueden registrar DDL, modificaciones, errores y esperas de bloqueo. La configuración permite escoger destino y formato; CSV facilita el análisis estructurado cuando el recolector de logs está habilitado.

El log_statement puede registrar texto de sentencias y parámetros; ciertos registros pueden revelar información sensible o contraseñas en claro. Registra lo necesario, limita acceso y retención, y decide por separado qué identidad y evento necesita una auditoría de negocio.

Ejemplo guiado

Configuración ilustrativa del servidor · PostgreSQL 18

```
# En postgresql.conf; no es una sentencia SQL
logging_collector = on
log_destination = 'csvlog'
log_statement = 'ddl'
```

Lectura

CSV requiere logging_collector. ddl registra sentencias de definición como CREATE, ALTER y DROP, no todas las modificaciones de filas. Cambiar esta configuración requiere revisar permisos y estrategia de rotación de logs.

Alcance y precisión

La captura de sentencias no prueba por sí sola quién autorizó una operación ni cubre los eventos de la aplicación. PostgreSQL advierte que los logs pueden contener datos sensibles; protégelos como datos del sistema.

Práctica

Define qué evento de préstamo debe poder reconstruirse, qué identificador de usuario lo vincula y qué dato no debe copiarse al log. Compara el registro de aplicación con el log del servidor.

Comprobación

Puedes identificar qué registra ddl, por qué csv necesita el recolector y qué riesgos presenta habilitar el registro indiscriminado.

Fuentes enlazadas: Error reporting y logging · PostgreSQL · Mantenimiento de logs · PostgreSQL

Tema 14 · Protección de datos y secretos

Objetivo

Reducir la exposición de datos mediante permisos precisos y tratar las credenciales como secretos operativos.

Explicación

Guarda solo los atributos que el servicio necesita, separa roles por función y concede acceso a columnas cuando el conjunto de datos lo permita. Los registros, volcados y copias también pueden contener datos personales y requieren controles de acceso y retención.

PostgreSQL ofrece autenticación SCRAM-SHA-256 entre sus métodos por contraseña. Evita incluir credenciales en scripts compartidos; si se usa el archivo de contraseñas de libpq, restringe su acceso según el sistema operativo y la guía oficial.

Ejemplo guiado

Conceder solo lectura de campos necesarios · PostgreSQL 18

```
CREATE ROLE consulta_publica NOLOGIN;
GRANT USAGE ON SCHEMA biblioteca TO consulta_publica;
GRANT SELECT (id_socio, nombre)
  ON biblioteca.socio TO consulta_publica;
```

Lectura

El rol puede leer dos columnas de socio, pero no recibe permiso de lectura de email ni de modificación en virtud de estas instrucciones. La autenticación y la provisión de credenciales se configuran por separado.

Alcance y precisión

El ejemplo de permisos no constituye una evaluación legal ni una política universal de privacidad. Verifica la finalidad, los plazos y los controles que exige la organización y la normativa aplicable al despliegue.

Práctica

Añade email a Socio y decide qué roles pueden leerlo. Revisa también si un dump, un log de consultas o un entorno de desarrollo podría copiar ese dato fuera de su finalidad.

Comprobación

Puedes describir qué columna queda fuera del permiso y cómo protegerías una credencial y una copia de datos de prueba.

Fuentes enlazadas: Privilegios por columna · PostgreSQL · Autenticación por contraseña · PostgreSQL · Archivo de contraseñas libpq · PostgreSQL

Tema 15 · Proyecto: operar un registro de préstamos

Objetivo

Integrar modelo, restricciones, recuperación, permisos y cambios versionados en un ejercicio verificable.

Explicación

Este esquema de práctica conserva socios, ejemplares y el historial de préstamos. La fecha de devolución puede quedar vacía mientras el préstamo está activo; un índice único parcial impide dos préstamos activos para el mismo ejemplar.

La solución necesita además una transacción para prestar, roles separados, migraciones revisadas, backup y un ensayo de restauración. El DDL implementa reglas de ejemplo y no sustituye requisitos de una biblioteca real.

Ejemplo guiado

Esqueleto integrador · PostgreSQL 18

```
CREATE TABLE socio (
  id_socio bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  nombre text NOT NULL, email text UNIQUE
);
CREATE TABLE ejemplar (
  id_ejemplar bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  codigo text NOT NULL UNIQUE
);
CREATE TABLE prestamo (
  id_prestamo bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  id_socio bigint NOT NULL REFERENCES socio(id_socio),
  id_ejemplar bigint NOT NULL REFERENCES ejemplar(id_ejemplar),
  fecha_prestamo date NOT NULL DEFAULT CURRENT_DATE, devuelto_en date,
  CHECK (devuelto_en IS NULL OR devuelto_en >= fecha_prestamo)
);
CREATE UNIQUE INDEX un_prestamo_activo
  ON prestamo (id_ejemplar) WHERE devuelto_en IS NULL;
```

Lectura

La clave foránea enlaza cada préstamo con sus entidades. Si se omite fecha_prestamo, CURRENT_DATE toma la fecha de inicio de la transacción. CHECK valida el orden entre fecha_prestamo y devuelto_en; el índice único parcial conserva el historial y limita a uno los préstamos sin fecha de devolución por ejemplar.

Alcance y precisión

El SQL aplica reglas deliberadamente escogidas para el ejercicio. Antes de producción faltarían gestión de errores, autorización, política de retención, observabilidad, límites de carga y un plan probado de continuidad.

Práctica

Entrega diagrama ER, migración inicial, transacción de préstamo, roles de lectura y escritura, comando de backup y acta de restauración en una base aislada.

Comprobación

Puedes demostrar cómo se rechaza una referencia inválida, un segundo préstamo activo y una devolución anterior a la fecha del préstamo.

Fuentes enlazadas: Restricciones · PostgreSQL · Índices parciales · PostgreSQL · Valores predeterminados · PostgreSQL · Fecha actual · PostgreSQL · Backup y restauración · PostgreSQL

Proyecto integrador · copia y restauración verificadas

Objetivo

Practica un plan de copia y recuperación en una instancia local o de pruebas. No ejecutes estas operaciones sobre producción: primero identifica el entorno y confirma que puedes restaurar sin sobrescribir datos ajenos.

Pasos de trabajo

1. Crea un esquema de prueba con un alumno y varias matrículas; anota la cantidad de filas y las restricciones que deben conservarse.

2. Genera una copia en formato adecuado y registra la versión del cliente y del servidor que utilizaste.

3. Restaura en una base nueva y aislada. Comprueba tablas, restricciones, índices, secuencias y permisos necesarios.

4. Simula una fila dañada solo en la copia de prueba y redacta cómo detectarías el problema antes de aceptar la restauración.

Comprobaciones

- La base restaurada contiene el número esperado de filas.

- Las claves y restricciones rechazan datos inválidos después de restaurar.

- La copia original permanece intacta y el procedimiento deja pasos repetibles.

Explicación final

Un archivo de copia no prueba por sí solo que el plan funciona. Documenta cuánto tardó la recuperación, qué permisos fueron imprescindibles y qué evidencia tendría que revisar otra persona antes de declarar el servicio recuperado.